

Intelligent Robot Project Proposal

April 2026

1 Requirements of Project Proposal

For your final project, you are expected to select a robot topic and formulate a concrete problem. Typical choices include the following:

- Select a **recent paper** from top conferences or journals such as RSS, CoRL, or RAL as your baseline. Analyze its **limitations, weaknesses, disadvantages, or future directions**, and propose your own method to improve or extend it. You are expected to implement your ideas and verify them through experiments.
- Select a **specific setting** (for example, robot manipulation, navigation, locomotion, or any new setting you define). You should identify or collect a suitable **dataset**, design your own **algorithm**, train your model, and evaluate its performance against appropriate **benchmarks**.
- Utilize some **existing robotics algorithms** to develop an interesting application. The modules of your application do not need to outperform state-of-the-art methods, but you must **integrate them effectively to create a coherent, functional system**.

This proposal should be concise, technically clear, and feasible within the project timeline.

You are required to write a proposal for your final project. There are several requirements for the proposal:

1. The proposal without reference should be no more than **two pages**, and you should write your proposal in LaTeX with RSS template ([here](#)).
2. The format of proposal is close to mini paper, so the content should include the following parts at least:

- **Project title.** Choose a concise and descriptive title that reflects the core focus of your work.
- **Overview or Abstract.** Provide a concise paragraph summarizing the entire proposal. This should include the problem you aim to address, the type of project you are conducting, the key methods or components you plan to use, and the main outcomes you expect to deliver.
- **Motivation and problem statement.** Clearly describe your topic and specify whether it is a research project or an application project. Explain the problem you intend to investigate and why it is meaningful or interesting. At this stage, high-level explanations are sufficient; detailed implementations are not required.
- **Literature review.** Provide a concise overview of the background and relevant prior work. Summarize the key ideas from the papers you cite and explain how they relate to your proposal. Ensure that all references are cited properly.
- **Technical plan.** State your plan for addressing the proposed topic.
 For research projects, describe the method or algorithm you aim to develop. **Mathematical formulations and diagrams of your model architecture** are preferred. If you plan to use existing implementations, specify which ones and how they will be incorporated. Indicate how you expect to improve or modify current methods. You do not need finalized solutions at this stage, but you should present a coherent direction for approaching the problem.
 For application projects, **diagrams of your system architecture and any preliminary demo materials** are preferred. List the **key features** of your application and explain how they differ from or improve upon existing applications, if relevant.
- **Expected outcomes.** Describe the results you expect to achieve. These may include algorithmic performance, system capabilities, evaluation metrics, or a functional demo, depending on your project type.
- **Potential risks and backup plan.** Identify possible obstacles, such as technical challenges, dataset limitations, or model training issues. Propose backup strategies that will allow you to continue making progress if your original approach becomes infeasible.
- **Timeline.** Provide a reasonable schedule that outlines major milestones and expected completion dates.
- **References.** List all works cited in the proposal using the standard RSS format. Ensure that the references are complete, correctly formatted, and limited to materials

directly relevant to your topic. Every in-text citation must appear in the list, and no unrelated or unused references should be included.

3. You should not reuse examples or materials taken directly from other courses or on-line sources; make substantive modifications to ensure the work is genuinely your own.

Plagiarism will make you fail in this project.

2 Idea Pool

Below are several potential project directions. You may choose to build upon one of these examples or design an entirely original solution. We highly encourage creativity; **projects that explore original ideas beyond these provided baselines will generally receive higher evaluations**. If you have any concerns about your project plan or need guidance, please consult with the course staff before finalizing your proposal.

2.1 Locomotion Sim2Sim

To verify the generalization capacity of pretrained locomotion policy, we sometimes transfer the policy to different simulation platform and validate its performance.

How to finish this project:

- Reproduce beyondmimic in isaac platform.
- Validate policy's performance based on this paper
- Transfer the policy to mujoco
- Validate policy's performance on mujoco platform
- Analysis the sim to sim capacity of the policy

Reference paper&code base: <https://beyondmimic.github.io/>

2.2 Multiple motion tracking

Existing motion tracking methods like beyondmimic can only tracking single motion pattern in one policy. We try to increase model scale and dataloader to track multiple motion patterns in one policy. How to finish this project:

- Reproduce beyondmimic in isaac platform.

- Validate policy's single motion tracking performance based on this paper
- Try to train the policy with multiple motion
- Validate policy's performance on multiple motion tracking.

Reference paper&code base: <https://beyondmimic.github.io/>

Suggestions: You could increase the MLP to bigger size and change the dataloader to learn. Sometimes, you only change the dataloader will have a good performance on multiple motion tracking.

2.3 General robot pick-up skills learning

Train a robot pick up policy can generalize to different environments.

- Download the robomimic github repo and carefully read the documentation
- Reproduce one of the pick up tasks like pickup cubes from robomimic
- Modify the task object/table/environment appearance for training
- Validate the policy's performance under different object or environment settings

Reference paper&code base: <https://robomimic.github.io/>

2.4 Quadruped robot parkour

To verify the agility and reinforcement learning capabilities of legged robots, we can train a quadruped robot to overcome complex obstacles like hurdles, gaps, and ramps using vision and proprioception.

How to finish this project:

- Reproduce a baseline parkour policy in a simulation platform like Isaac Gym.
- Validate the robot's performance on standard obstacle tracks (e.g., walking over small blocks).
- Design a new terrain or obstacle type (e.g., increasing the gap width, or adding moving obstacles).
- Fine-tune or retrain the policy to adapt to the newly designed environment.

- Evaluate the success rate and analyze the failure cases (e.g., falling or colliding).

Reference paper&code base: <https://extreme-parkour.github.io/>

Suggestions: Training a robot to jump over high obstacles from scratch can be very hard. You can start with simple, low obstacles and use "curriculum learning" (gradually increasing the difficulty of the terrain during training) to help the policy converge faster.

2.5 Language-guided motion generation

Instead of traditional object manipulation, this project focuses on the kinematic generation of character or humanoid robot motions based on natural language instructions. You will explore how discrete motion representations can be controlled via text.

How to finish this project:

- Download the MoConvQ codebase and set up the required environment and datasets (e.g., HumanML3D).
- Reproduce the baseline model using the provided pre-trained weights to generate motions from standard text prompts.
- Create a customized subset of text-motion pairs to introduce a new stylistic movement, or modify the text condition embedding/sampling strategy during inference.
- Fine-tune the pre-trained model on your customized data or test the zero-shot capabilities of your modified sampling strategy.
- Validate the policy's performance by visually comparing the generated 3D motions and evaluating standard metrics (like Frechet Inception Distance for motions).

Reference paper&code base: <https://moconvq.github.io/>

Suggestions: Do not train the motion quantization and generation models from scratch, as it requires massive computational resources. Use the provided pre-trained weights and focus on the fine-tuning process or modifying the generation conditions. Reserve full training from scratch only for cases where the target performance cannot be reached otherwise.

2.6 Vision-based robot navigation

Navigate an indoor environment using only first-person visual observations to reach a specific target.

How to finish this project:

- Install the required visual navigation simulator and clone the ObjectReact repository.
- Reproduce the baseline ObjectGoal navigation policy using the provided pre-trained weights.
- Modify the environment’s reward function (e.g., add a new penalty for getting too close to non-target objects to improve safety) OR introduce visual sensor noise (e.g., simulate imperfect object detection bounding boxes).
- Resume training or fine-tune the agent with your specific modifications.
- Evaluate the agent’s Success rate and Success weighted by Path Length (SPL), analyzing how your modifications impacted its navigation strategy and robustness.

Reference paper&code base: <https://object-react.github.io/>

Suggestions: This repository provides a solid starting point for visual navigation. You can complete a large portion of this project by understanding the provided codebase, tweaking the reward calculation logic, or modifying the observation inputs without needing to implement complex reinforcement learning algorithms from scratch.